

Tworzenie aplikacji webowych

Shiny for Python

STUDIA PODYPŁOMOWE

Data Science w zastosowaniach biznesowych

Uniwersytet Warszawski, Wydział Nauk Ekonomicznych



UNIwersYTET
WARSAWski



WYDZIAŁ NAUK
EKONOMICZNYCH

Ewa Weychert, PhD

Copyright © 2025 WNE UW. All rights reserved.

Spis Treści

- 1 Dashboard vs. aplikacja webowa
- 2 Dlaczego Shiny?
- 3 Streamlit vs Shiny
- 4 Jak działa Shiny for Python?
- 5 Możliwości Shiny for Python
- 6 Shiny Core vs Shiny Express
- 7 Podsumowanie

Czym różni się dashboard od web application?

Dashboard (pulpit nawigacyjny)

- Prezentuje dane wizualnie (wykresy, tabele, KPI).
- Służy do monitorowania, analizy i podejmowania decyzji.
- Ograniczona interakcja — użytkownik ogląda i filtruje dane.
- Raczej nie modyfikuje danych, tylko je eksploruje.
- *Przykład: panel administratora z wykresami sprzedaży.*

Web application (aplikacja webowa)

- Pełnoprawna aplikacja działająca w przeglądarce.
- Może mieć wiele funkcji: rejestracja, edycja danych, zarządzanie procesami.
- Wysoki poziom interakcji — użytkownik dodaje, edytuje, usuwa dane.
- *Przykład: sklep internetowy.*

Podsumowanie

Dashboard = miejsce, gdzie *pokazujesz*, co znalazłeś.

Web application = miejsce, gdzie *zbierasz*, co musisz przeanalizować.

Dlaczego Django, a nie Shiny?

Django = pełny framework webowy (Python)

- Oddziela logikę (Python) od interfejsu (HTML).
- Daje dużą kontrolę — ale wymaga więcej kodu.
- Trzeba rozumieć: views, templates, routing, forms, models.

Shiny = szybka wizualizacja

- Idealne do wykresów i interakcji z danymi.
- Mniej kodu, mniejsza elastyczność.

Potrzebujesz...	Użyj:
Tylko wykresów i interakcji	Shiny
Pełnej aplikacji z bazą, użytkownikami, API	Django
Kontroli nad frontem, backendem i autoryzacją	Django

Czym jest Shiny for Python?

- **Framework** do tworzenia interaktywnych aplikacji webowych w Pythonie.
- Stworzony przez **Posit** (twórców R Shiny).
- Działa **reaktywnie** — przelicza tylko to, co się zmienia (lepsze niż Streamlit).

Uwaga

Część uczestników kursu korzysta z języka R oraz pakietu Shiny. Profesor Wójcik również je rekomendował.

Streamlit vs Shiny for Python — porównanie

Cecha	Streamlit	Shiny for Python
Krzywa nauki	Bardzo łagodna	Nieco wyższa (reaktywny)
Model programowania	Imperatywny (liniowy)	Deklaratywny, reaktywny
Elastyczność UI	Ograniczona	Bardziej elastyczne UI
Integracja HTML/CSS	Ograniczona	Pełna kontrola
Reaktywność	Ręczna	Wbudowana
Wydajność	Dobra (proste appl.)	Lepsza skalowalność
Wsparcie	Duża społeczność	Nowe, rozwijane przez Posit
Zastosowania	Dashboardsy, prototypy	Złożone appl. reaktywne

- Jeśli zależy Ci na szybkim MVP lub dashboardzie — wybierz **Streamlit**.
- Jeśli planujesz złożoną, interaktywną aplikację — **Shiny** zaoferuje więcej przewagi.

Jak działa Shiny for Python?

- Shiny for Python opiera się na **3 głównych elementach**:

- 1 **UI (User Interface)** — interfejs użytkownika

Tworzony przy użyciu funkcji z `shiny.ui`.

Można dodawać kontrolki (suwak, przyciski) i miejsca na wyniki (wykresy, tekst).

- 2 **Server** — logika aplikacji

Opisuje, jak reagować na dane wejściowe.

Używa dekoratorów `@output()` i `@render.*`.

- 3 **App** — połączenie UI i servera

Reaktywność w Shiny

Jak działa reaktywność?

- 1 Shiny monitoruje dane wejściowe (np. zmiana suwaka).
- 2 Automatycznie przelicza dane wyjściowe (wykresy, teksty) bez odświeżania strony.

Przykład z suwakiem:

- 1 Masz suwak (wybierasz liczbę 0–100).
- 2 Przesuwasz suwak z 50 na 75.
- 3 Shiny zapisuje nową wartość.
- 4 Automatycznie uruchamia kod zależny od suwaka.
- 5 Zmienia tylko to, co trzeba — bez przeładowania.

Prosta analogia

Masz kalkulator, wpisujesz $2 + 2$.
Zmieniasz jedną liczbę na 5
→ wynik od razu zmienia się na 7.
Nie trzeba klikać „Oblicz”.

Reaktywny = reaguje na zmianę.
Gdy dane wejściowe się zmieniają,
automatycznie zmieniają się
dane wyjściowe.

Przykład w R Shiny

```
library(shiny)

ui <- fluidPage(
  sliderInput("n", "N", min = 0, max = 100, value = 20),
  verbatimTextOutput("txt")
)

server <- function(input, output, session) {
  output$txt <- renderPrint({
    paste("n*2 is", input$n * 2)
  })
}

shinyApp(ui = ui, server = server)
```

Przykład w Shiny for Python

```
from shiny import App, ui, render, output

app_ui = ui.page_fluid(
    ui.input_slider("n", "N", 0, 100, 20),
    ui.output_code("txt")
)

def server(input, output, session):
    @output()
    @render.code
    def txt():
        return f"n*2 is {input.n() * 2}"

app = App(app_ui, server)
```

Jak uruchomić aplikację:

- `cd dashboard`
- `shiny run -reload app.py`
- Otwórz: `http://127.0.0.1:8000`

Możliwości Shiny for Python

- Dynamiczne formularze i wykresy.
- Obsługa wykresów: Plotly, matplotlib, pandas.
- Łatwa integracja z danymi (CSV, bazy danych).

Cecha	Shiny for Python	Django
Typ frameworka	Interaktywne appl. webowe	Pełny framework MVC
Zastosowanie	Dashboardy, data science	Strony, REST API
Architektura	Reaktywna	Request-response
UI	Python (bez HTML)	HTML/CSS/Jinja2
Routing	Brak (1 appl. = 1 UI)	Wbudowany routing
Dane	Pandas, NumPy, Plotly	ORM, SQL
Skalowalność	Mniejsza	Wysoka

Dwa tryby Shiny for Python

Cecha	Shiny Core	Shiny Express
Struktura	Oddzielne ui i server	Brak server, kod liniowy
Szybkość pracy	Wolniejszy start	Błyskawiczne prototypowanie
Skalowalność	Wysoka	Ograniczona (dobre dla MVP)
Krzywa nauki	Wyższa	Bardzo łagodna

Shiny Express — jak działa?

- Brak funkcji `App(ui, server)`.
- Brak `@output`, `@server` — wszystko dzieje się w jednym miejscu.
- Idealny do szybkiego pisania dashboardów i prototypów.

Kiedy wybrać który tryb?

Scenariusz	Wybierz. . .
Prototyp aplikacji	Express
Duży, skalowalny projekt	Core
Przechodzisz ze Streamlit	Express
Znasz R Shiny	Core
Uczysz się Shiny po raz pierwszy	Express (na start)

Jak przejść z Express do Core?

- 1 Dodaj funkcję `server(input, output, session)`.
- 2 Przenieś wszystkie dekoratory `@render` do środka `server`.
- 3 Zbuduj osobny `ui` jako `layout`.
- 4 Na końcu użyj: `app = App(ui, server)`.

Shiny Core — jak działa?

- Podział kodu na:
 - `ui` — logika wyświetlania,
 - `server` — logika działania.
- Przypomina styl z R Shiny.
- Umożliwia skalowanie i rozbudowę aplikacji.

Podsumowanie

- **Shiny Express:** idealny do nauki i szybkich dashboardów.
- **Shiny Core:** profesjonalny, rozbudowany, gotowy na produkcję.
- Oba tryby są aktywnie rozwijane i będą współistnieć.
- Dokumentacja Posit wspiera oba style.

Kiedy wybrać Shiny?

- Gdy chcesz szybko zbudować interaktywny dashboard.
- Gdy pracujesz z danymi naukowymi (data science, analizy).
- Gdy chcesz uniknąć pisania front-endu (HTML/CSS/JS).

Literatura i zasoby

- <https://shiny.posit.co/py/>
- <https://github.com/posit-dev/py-shiny>
- <https://towardsdatascience.com/understanding-the-two-faces-of-shiny>
- <https://github.com/posit-dev/streamlit-shiny-dash>